Text Processing

Part 2

Session 10

Agenda:

- Intro to Text Processing
- Basic Text Manipulation commands:
  - “cut” command
  - “sort” command
  - “uniq” command
- Searching with “grep” command
- Regex and Pattern matching:
  - What is Regex?
  - BRE syntax
  - **ERE syntax (how it differs from BRE)**
  - **PCRE syntax and usage**
- “sed” command
- “awk” command

We’ll continue from here

Recap on Regex

Recap on Regex:

- Regular Expressions (Regex) are sequences of characters used to search, find, and manipulate text using patterns.

Regex comes in a few versions, main ones are:

1. **BRE**: Basic Regular Expressions
2. **ERE**: Extended Regular Expressions
3. **PCRE**: Perl-Compatible Regular Expressions

Recap on Regex:

Anchors:

^	Matches lines starting with the pattern
$	Matches lines ending with the pattern

Alternation:

\ |	Matches one pattern OR another

Recap on Regex:

Repetition:

.	Matches any single character
?	Matches zero or one of the previous item
*	Matches zero or more of the previous item
+	Matches one or more of the previous item
\{n\}	Matches exactly n times of the previous item
\{n, \}	Matches at least n times of the previous item
\{, m\}	Matches at most m times of the previous item
\{n, m\}	Matches from n to m times of the previous item

Recap on Regex:

Character classes:

Type	Function	Syntax
Any of a set	Match a, b, or c	[abc]
A range	Match numbers between 1 and 8	[1-8]
Negation	Match anything except a, b, or c	[^abc]

Grouping and capturing:

\(pattern\)	\1 \2 \3

Types of Special characters

Metacharacters that are special by default:	Characters that become special when escaped by backslash \:
• .	• \\\|
• *	• \+
• ^	• \?
• $	• \{ \}
• [ ]	• \(\)

But why we don’t make it simpler?

Extended Regular Expression

Extended Regular Expressions (ERE)

- It's considered as an upgrade to BRE.
- The main difference is that now all metacharacters are special **by default**.

.	*	^	$	[ ]	|	+	?	{ }	( )

- And to use them as literals by escaping them with backslash \

Other than that, it works the same way as BRE

Extended Regular Expressions (ERE)

Using ERE in grep:

```
❯ grep -E '^[0-9]{4}( [0-9]{4}){3}' mixed.txt
1720 1806 5148 9051
7721 2420 1480 9025
4800 4370 2056 7300
5234 5796 7762 4410
9018 1074 5834 6896
4754 5660 6735 1522
7817 7468 4799 0417
6192 4508 4962 4218
```

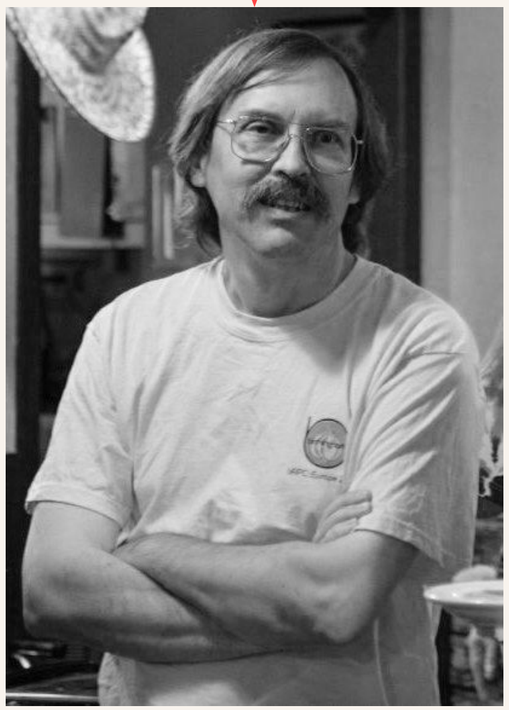
Perl Compatible Regular Expression

Perl Compatible Regular Expressions (PCRE)

PCRE is a regex engine modeled after Perl's regex implementation from Larry Wall (creator of Perl programming language).

Many tools and programming languages use PCRE or PCRE-like syntax:

- grep (with -P option)
- Java
- Perl *-of course-*
- Python
- PHP
- .NET framework



Perl Compatible Regular Expressions (PCRE)

- PCRE significantly extends the capabilities of POSIX BRE and ERE.
- It has many features that both BRE and ERE don't have like:
  i. Shorthand character classes
  ii. Lookahead
  iii. Lookbehind
  iv. Conditional patterns
  v. Non-greedy quantifiers
  vi. Named groups

I. Shorthand character classes

- Similar to POSIX character classes but better and has more capabilities

`\d`	Matches a digit, same as [0-9]
`\D`	Matches a non-digit
`\w`	Matches an alphanumeric character, including "_"
`\W`	Matches a non-alphanumeric character, excluding "_"
`\s`	Matches a whitespace character (space, tab, newline, form feed)
`\S`	Matches anything BUT a whitespace
`\b`	Matches a word boundary
`\A`	Matches the beginning of a string (but not an internal line)
`\Z`	Matches the end of a string (but not an internal line).

I. Shorthand character classes

```
❯ grep -P '\w+' grocery
Bread    2020
Oranges 1015
Apple    512
Mac&cheese      390
```

Matches alphanumeric characters

```
❯ grep -P '\D+' grocery
Bread    2020
Oranges 1015
Apple    512
Mac&cheese      390
```

Matches non-digit characters

II. Lookahead

Type	Function	Syntax
Positive lookahead	Match pattern followed by x	`pattern(?=x)`
Negative lookahead	Match pattern not followed by x	`pattern(?!x)`

III. Lookbehind

Type	Function	Syntax
Positive lookbehind	Match pattern preceded by x	`(?<=x)pattern`
Negative lookbehind	Match pattern not preceded by x	`(?<!x)pattern`

II. Lookahead

Notice that only the numbers are matched

```
❯ grep -P '\d+(?= USD)' prices.txt
10 USD
25 USD
40 USD
75 USD
120 USD
36 USD
```

III. Lookbehind

Same thing here

```
❯ grep -P '(?⇐\d )EGP' prices.txt
54 EGP
89 EGP
150 EGP
230 EGP
310 EGP
58 EGP
```

IV. Conditional patterns

- It matches alternate patterns depending on whether a condition is True or False

```
'(?(condition)TruePattern|FalsePattern)'
```

Checks if a pattern/lookahead/lookbehind exists or not

If true, match this pattern

If not, match this pattern instead

IV. Conditional patterns

```
❯ grep -P '\d{4}(?(?=-)-\d{2}-\d{2}|/\d{2}/\d{2})' dates.txt
2024-03-15
2023/12/25
2022-07-04
2021/11/30
2020-05-18
2019/08/22
2018-03-14
2017/11/09
2016-06-27
2015/09/03
```

Matches two different date formats

V. Non-greedy (lazy) quantifiers

- By default, quantifiers are greedy
- They match as many times as possible

*
+
{n,m}

Matched as many “s” as possible

```
❯ grep -P 'jars+' words
jars
jarsssss
```

V. Non-greedy (lazy) quantifiers

- Unlike in BRE and ERE, PCRE gives you the option to specify whether you want them greedy or lazy
- Lazy quantifiers are set by a `?` after the quantifier, and it matches as few times as possible

`*?`
`+?`
`{n,m}?`

Matched as few “s” as possible

```
❯ grep -P 'jars+?' words
jars
jarssssss
```

VI. Named groups

- Instead of calling captured groups with their numbers, you have the option to specify names to each group

`'(?<name>pattern)'`	`'\k<name>'`
Capturing a group and specifying its name	Calling the captured group with its name

VI. Named groups

```
❯ grep -P '(?<year>\d{4})/(?<month>\d{2})/(?<day>\d{2})' dates.txt
2023/12/25
2021/11/30
2019/08/22
2017/11/09
2015/09/03
```

Now the naming is more descriptive

Hands on

1

Search in “emails.txt” for username “mike” in emails that doesn’t have “gmail” in its domain

john@example.com

username domain

Solution:

```
grep -P 'mike(?!@gmail)' emails.txt
```

```
❯ grep -P 'mike(?!@gmail)' emails.txt
mike@hotmail.com
mike@yahoo.com
mike@aol.com
```

Let’s take a Break!

“sed” command

“sed” command

- “`sed`” stands for stream editor and it can perform lots of functions on file like **searching**, **find and replace**, **insertion** or **deletion**.
- The most common use of “`sed`” command is for substitution (find and replace).
- By using “`sed`” you can edit files even without opening them, which is much quicker way to find and replace something in file, than opening the file with a text editor and changing it
- By default, “`sed`” uses BRE.
- With `-E`, it uses ERE.

Basic syntax: `sed [options] script file`

Options:

`-e script`	Adds script to the commands to be executed (can be done multiple times)
`-n`	Suppress automatic printing of each processed line
`-i`	Edit files in place (`-i.bak` additionally creates a backup file)
`-f file`	Add the contents of file to the commands to be executed

Scripts can vary in the way they're written and their internal commands depending on what you want sed to do, but the flow is simple:

1. "`sed`" reads one line into memory (called pattern space)
2. Runs your script
3. Prints the result (unless suppressed)
4. Moves to next line

Substitution command:

- The syntax for the substitution command works like this:

```
sed -option 's/pattern/replacement/[flags]' filename
```

`pattern:`	The desired pattern (regex) we want to replace
`replacement:`	What will the pattern be replaced by
`flags:`	Modify how the substitution behaves (optional)

```
❯ cat cat.txt
I love cats!
❯ sed 's/cat/dog/' cat.txt
I love dogs!
```

Substitution command:

Note: Conventionally, the delimiter is "/", but it can be -*literally*- any character, as long as you escape it with a backslash if it's used in the pattern as a literal

```
❯ sed 's/cat/dog/' cat.txt
I love dogs! My cats are cute.
All dogs are cute.
❯ sed 's:cat:dog:' cat.txt
I love dogs! My cats are cute.
All dogs are cute.
❯ sed 'sac\atadoga' cat.txt
I love dogs! My cats are cute.
All dogs are cute.
```

Nobody should do something like this.
But technically speaking, it's possible!

Substitution command:

Flags:

'g'	Replace all matches in pattern space
'*n*' (stands for any number)	Replace only the nth occurrence per line

```
❯ sed 's/cat/dog/' cat.txt
I love dogs! My cats are cute.
All dogs are cute.
❯ sed 's/cat/dog/g' cat.txt
I love dogs! My dogs are cute.
All dogs are cute.
❯ sed 's/cat/dog/2' cat.txt
I love cats! My dogs are cute.
All cats are cute.
```

Substitution command:

Flags:

`'p'`	Used with -n option to print only modified lines
`'w file'`	Write modified line to `file`

```
❯ cat fruits
Banana
Grape
Apple
orange
apple
kiwi
kiwi
orange
❯ sed -n 's/orange/pommegrante/p' fruits
pommegrante
pommegrante
```

```
❯ sed 's/orange/pommegrante/w newfruits' fruits
Banana
Grape
Apple
pommegrante
apple
kiwi
kiwi
pommegrante
❯ cat newfruits
pommegrante
pommegrante
```

Substitution command:

- There might be cases where you want to search for the pattern and replace that pattern by adding some extra characters to it. In such cases & comes in handy. The & represents the matched string.

```
[sed]$ cat data
one two three, one two thrEE one one
four THree two one
one hundred
[sed]$ sed 's/one/{&}/g' data
{one} two three, {one} two thrEE {one} {one}
four THree two {one}
{one} hundred
[sed]$ sed 's/one/^&^/g' data
^one^ two three, ^one^ two thrEE ^one^ ^one^
four THree two ^one^
^one^ hundred
[sed]$ sed 's/one/(^&^)/g' data
(^one^) two three, (^one^) two thrEE (^one^) (^one^)
four THree two (^one^)
(^one^) hundred
```

Substitution command:

- In other cases, we just need to change the order of some groups in the pattern or change the format.
- Here we can capture those groups and use them as we like in the replacement pattern

```
❯ cat names
Johnson, Alice
Smith, Bob
Brown, Charlie
Elkhayat, Abdulrahman
❯ sed -E 's/([A-Za-z]+), ([A-Za-z]+)/\2 \1/' names
Alice Johnson
Bob Smith
Charlie Brown
Abdulrahman Elkhayat
```

Recalling previously captured groups

\1

\2

Deletion command:

- Deletion command deletes lines as following:

`'d'`	Delete all lines
`'nd'`	Delete the nth line
`'n,md'`	Delete line in range from `n` to `m`
`'$d'`	Delete last line
`'n,$d'`	Delete from the nth line to the last line
`'/pattern/d'`	Delete lines matching `pattern`

Deletion command:

Example:

```
❯ cat lines.txt
One
Two
Three
Four
Five
❯ sed '2d' lines.txt
One
Three
Four
Five
❯ sed '2,4d' lines.txt
One
Five
❯ sed '$d' lines.txt
One
Two
Three
Four
❯ sed '2,$d' lines.txt
One
❯ sed '/ree/d' lines.txt
One
Two
Four
Five
❯ sed 'd' lines.txt
```

No output (all lines are deleted)

Insertion:

- You can insert lines as following:

`'/pattern/a words'`	Inserts `words` a new line after the matched pattern
`'/pattern/i words'`	Inserts `words` a new line before the matched pattern
`'p'`	When used without -n, it duplicates all lines

```
❯ sed '/love/ a -Hello-' cat.txt
I love cats! My cats are cute.
-Hello-
All cats are cute.
❯ sed '/love/ i -Hello-' cat.txt
-Hello-
I love cats! My cats are cute.
All cats are cute.
```

```
❯ sed 'p' cat.txt
I love cats! My cats are cute.
I love cats! My cats are cute.
All cats are cute.
All cats are cute.
```

Searching:

- You can make sed command to work as similar to grep command

grep command	sed's version
`grep 'word' file.txt`	`sed -n '/word/p' file.txt`
`grep -v 'word' file.txt`	`sed -n '/word/!p' file.txt`

```
❯ grep 'love' cat.txt
I love cats! My cats are cute.
❯ sed -n '/love/p' cat.txt
I love cats! My cats are cute.
❯ grep -v 'love' cat.txt
All cats are cute.
❯ sed -n '/love/!p' cat.txt
All cats are cute.
```

Hands on

2

In “store.txt”, replace any “USD” with “$” (in the same file)

Solution:

```
sed -i.bak 's/USD/$/' store.txt
```

Used .bak in case anything went wrong

```
❯ sed -i.bak 's/USD/$/' store.txt
❯ cat store.txt
Object          Price   Currency
Laptop          1000    $
Coffee-Mug      13      $
Desk-Chair      245     EUR
Smartphone      699     GBP
Backpack        80      CAD
Headphones      150     EUR
Notebook        4       $
Water-Bottle    18      AUD
Mouse           30      $
Keyboard        89      GBP
Monitor         329     EUR
Desk-Lamp       46      $
Tablet          449     CAD
Book            25      $
Pen-Set         9       GBP
Sunglasses      159     EUR
Watch           299     CHF
Speaker         90      $
```

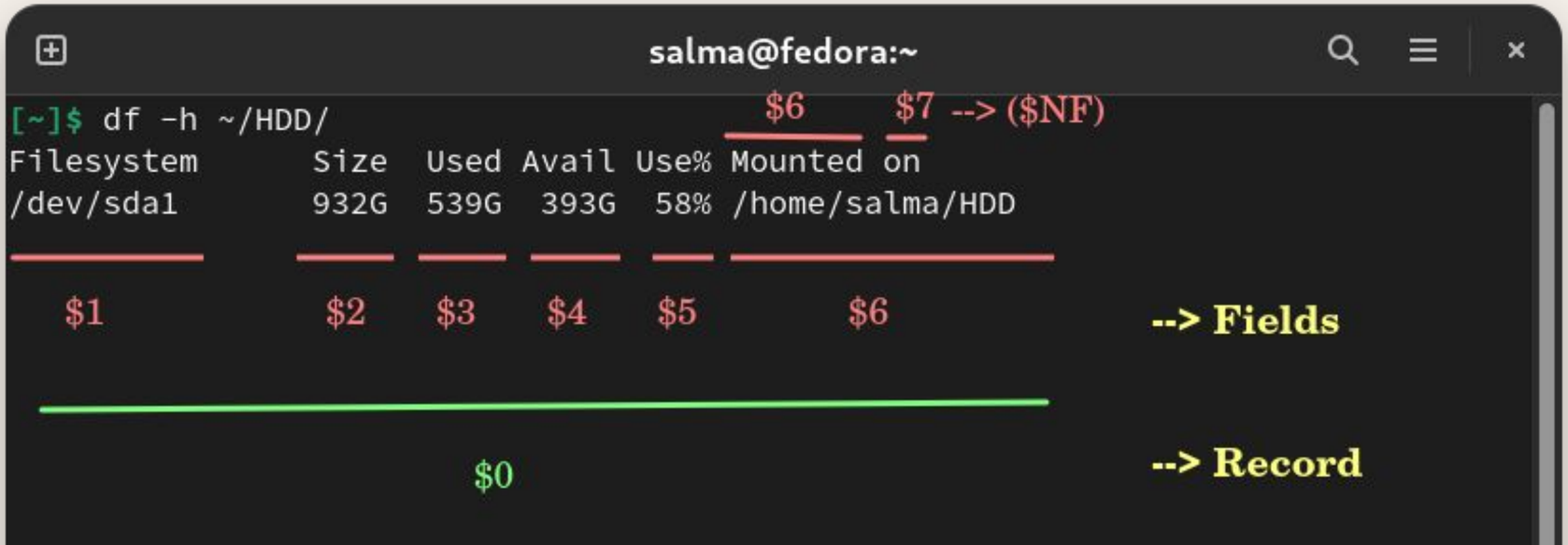
“awk” command

“awk” command

- It is named after its creators:
  - Alfred Aho
  - Peter Weinberger
  - Brian Kernighan
- **AWK** is a small programming language designed for easy manipulation of structured text.
- Like common programming language, Awk has variables, conditionals, loops, arithmetic and string operators.
- For regex patterns, **AWK** uses ERE by default

Records and Fields

- The input data is divided into **records** and **fields**. Awk operates on one record at a time until the end of the input is reached. Records are separated by a character called the **record separator**.
- The default **record separator** is the **newline** character
- Records consist of fields which are separated by the **field separator**. By default, the value of the **field separator** is **any number** of **space** or **tab** characters.
- The fields in each record are referenced by the dollar sign ($) followed by **field number**, beginning with 1, The last field can also be referenced with the special variable $*NF*. The entire record can be referenced with $*0*.



Records and Fields

- To change the default **field separator** we use **-F** option followed directly by the separator to be specified.

```
❯ cat grades_tab
Name    Math    Physics Chemistry       Biology History
Alice   68      92      63      72      53
Bob     69      84      75      85      94
Charlie 80      91      58      64      57
David   51      95      85      59      86
Eve     93      55      90      66      83
❯ awk '{print $1}' grades_tab
Name
Alice
Bob
Charlie
David
Eve
```

```
❯ cat grades.csv
Name,Math,Physics,Chemistry,Biology,History
Alice,88,56,86,87,72
Bob,81,68,76,88,98
Charlie,61,50,75,95,88
Ethan,75,74,87,62,57
❯ awk -F, '{print $1}' grades.csv
Name
Alice
Bob
Charlie
Ethan
```

Command rules

```
awk '/pattern/ {action}' file
```

- Awk reads the input files one line at a time.
- For each line, it matches with given pattern in the given order, if matches performs the corresponding action.
- If no pattern matches, no action will be performed.
- Either search pattern or action are optional, But not both.
- If the search pattern is not given, then Awk performs the given actions for each line of the input.
- If the action is not given, print all that lines that matches with the given patterns which is the default action.
- Empty braces without any action does nothing. It won't perform default printing operation.
- Each statement in Actions should be delimited by semicolon (;).

Command rules

This is the file will work on for this command:

```
❯ cat employees_tab
ID      FirstName      LastName       Department      Salary  JoinDate
101     John           Doe            Engineering     75000   2021-03-15
102     Jane           Smith          Marketing       65000   2020-08-22
103     Bob            Johnson        Engineering     82000   2019-11-30
104     Alice          Brown          Marketing       70000   2022-01-10
105     Charlie        Wilson         Sales           60000   2023-05-18
107     Eve            Davis          HR              55000   2020-02-28
```

Command rules

Pattern and Action

Action without Pattern

Pattern without Action

```
❯ awk '/Engineering/{print $2, $3 "\t" $4}' employees_tab
John Doe        Engineering
Bob Johnson     Engineering
❯ awk '{print $2 "\t" $4}' employees_tab
FirstName       Department
John    Engineering
Jane    Marketing
Bob     Engineering
Alice   Marketing
Charlie Sales
Eve     HR
❯ awk '/Engineering/' employees_tab
101     John            Doe             Engineering     75000   2021-03-15
103     Bob             Johnson         Engineering     82000   2019-11-30
```

Command rules

- By default, patterns are matched against each line (record)
- To make them match against specific fields, we specify the field and then use "~" operator before the pattern

`$n ~/pattern/`	Matches where `pattern` exists in the nth field
`$n !~/pattern/`	Matches where `pattern` doesn't exist in the nth field

```
❯ awk '$2 ~/^J/ {print $2 "\t" $4}' employees_tab
John    Engineering
Jane    Marketing
❯ awk '$2 !~/^J/ {print $2 "\t" $4}' employees_tab
FirstName       Department
Bob     Engineering
Alice   Marketing
Charlie Sales
Eve     HR
```

Command rules

- You can also make comparisons like <, > or ==

```
❯ awk '$5 > 60000 {print $2, $3}' employees_tab
FirstName LastName
John Doe
Jane Smith
Bob Johnson
Alice Brown
```

Command rules

- Awk includes the following special patterns:

BEGIN	Used to perform actions before records are processed
END	Used to perform actions after records are processed

```
❯ awk 'BEGIN {print "═══════════════════════════════════"};
{print $2, $3};
END {print "═══════════════════════════════════"}' employees_tab
═══════════════════════════════════
FirstName LastName
John Doe
Jane Smith
Bob Johnson
Alice Brown
Charlie Wilson
Eve Davis
═══════════════════════════════════
```

Built-in variables

Variables that can be defined and changed	
`FS`	Field separator of the original file
`RS`	Record separator of the original file
`OFS`	Field separator of the output only
`ORS`	Record separator of the output only
Read-only variables that hold a specific value	
`NF`	Number of fields in current record
`NR`	The number of the current record (global)
`FNR`	The number of the current record (in current file)
`FILENAME`	The name of the input file that is currently processed

Built-in variables

```
❯ awk 'BEGIN {OFS=" : "}; {print NR,FNR}' employees_tab employees.csv
1 : 1
2 : 2
3 : 3
4 : 4
5 : 5
6 : 6
7 : 7
8 : 1
9 : 2
10 : 3
11 : 4
12 : 5
13 : 6
14 : 7
15 : 8
```

Global run

First file

Second file

If-statements

- Like any regular programming language, Awk has if-statements.
- Works like... ummm, a **regular if-statement**

```
❯ awk '{if ($5 > 70000) print "Rich"
quote> else if ($5 > 60000) print "Middle class"
quote> else print "poor"}' employees_tab
Rich
Rich
Middle class
Rich
Middle class
poor
poor
```

Expressions

- Also, like any regular programming language, Awk has expressions.
- Nothing special, just regular variable assignments and mathematical expressions

```
❯ awk 'BEGIN {max = 0}; /[0-9]+/ {if ($5 > max) max = $5};
quote> END {print max}' employees_tab
82000
```

Hands on

3

Calculate the average grade over two classes (class-1 and class-2)

Solution:

```
❯ awk -F',' 'BEGIN {sum = 0}; /[0-9]+/ {sum += $2};
END {print "Average: " sum/(NR-2)}' class-1.txt class-2.txt
Average: 85.75
```

That’s it